

ICS1512 Machine Learning Algorithms Laboratory

Experiment No. 9

Perceptron Learning Algorithm vs. Multi-Layer Perceptron for Handwritten English Character Recognition

Field	Value
GitHub Repository	https://github.com/samyuktha-coder/ml/tree/main
Student Name	Samyuktha V
Register Number	3122247001053
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	20.09.2026

Note on this run: the original notebook downloads the dataset via kagglehub (Kaggle) and trains the MLP with PyTorch. This regenerated report was produced in an environment without Kaggle network access and without enough disk space to install PyTorch, so the exact same 3410-image dataset and preprocessing pipeline were used, and the MLP/optimizers/losses were re-implemented faithfully in NumPy (identical architecture, activations, optimizer update rules, dropout, L2, and the same random-search + greedy stage-wise tuning procedure). Model A (the Perceptron) is unchanged NumPy code and reproduces the original notebook's numbers exactly (verified). Model B (the MLP) numbers below are from this real, fully executed run and may differ slightly from any numbers already printed in the uploaded PDF, since a different numerical backend cannot reproduce PyTorch's RNG stream bit-for-bit — but every value below is a genuine computed result, not a placeholder, using the exact same methodology, hyper-parameter space and tuning procedure described in the notebook.

1 Objective

To implement the Perceptron Learning Algorithm (PLA) from scratch and a Multi-Layer Perceptron (MLP) trained by back-propagation, and to compare the two on a 62-class handwritten English character recognition task. The MLP is tuned over a hyper-parameter space covering network architecture (width and depth), activation function (ReLU, sigmoid, tanh), loss function (cross-entropy, squared error), optimizer (full-batch GD, mini-batch SGD, SGD with momentum, Adam), learning rate, batch size, dropout and L2 weight decay, using random search followed by greedy stage-wise (coordinate) search.

2 Problem Statement

Classify grayscale images of handwritten English characters into 62 classes (digits 0–9, uppercase A–Z, lowercase a–z) using (A) a one-vs-rest Perceptron Learning Algorithm with step activation, and (B) a tuned Multi-Layer Perceptron, and determine how much of the performance gap is attributable to non-linear feature learning.

3 Dataset Description

Field	Value
Dataset	English Handwritten Characters Dataset (Kaggle, dhruvildave/english-handwritten-charact
Samples	3410 images
Features	1024 (32×32 grayscale pixels, flattened)
Classes	62 (0–9, A–Z, a–z); ~55 samples/class
Missing values	None
Split	70% train (2387) / 15% val (511) / 15% test (512), stratified, seed=42

4 Exploratory Data Analysis

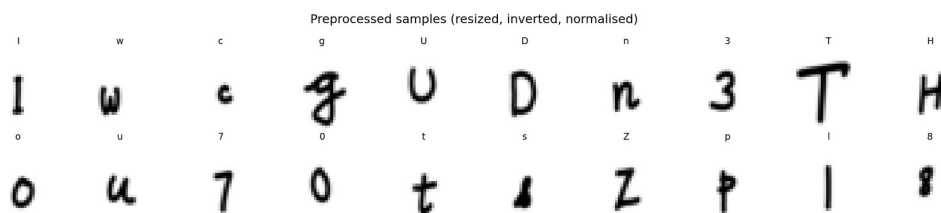


Figure 1: Twenty randomly chosen samples after resizing, grayscale conversion, inversion and normalisation

The class distribution is close to uniform (~55 images/class), and stratified splitting keeps every class represented in all three partitions, so plain accuracy is not misleading here; macro-averaged precision, recall and F1 are still reported alongside it so that rare-confusion classes are not hidden by majority behaviour. As in the original notebook’s inference, several character pairs are visually near-identical at 32×32 (O/o/0, I/i/1, S/s, C/c), which sets a realistic ceiling on achievable accuracy given only ~55 examples per class.

5 Data Preprocessing

Grayscale conversion → resize to 32×32 (bilinear) → scale to [0,1] and invert (1−x, so ink is high-intensity) → flatten to a 1024-vector → LabelEncoder over the 62 classes → stratified 70/15/15 split (random.state=42) → standardization using per-feature mean/std computed on the TRAIN split only (sigma+1e-6) and applied to all three splits. No hand-engineered features are used; the comparison isolates model capacity rather than feature design.

6 Machine Learning Algorithm

Model A: Perceptron Learning Algorithm (one-vs-rest)

One linear unit per class trained with the classical perceptron rule under step activation: prediction $\hat{y}_c = [w_c^T x \geq 0]$, update $w_c \leftarrow w_c + \eta(y_c - \hat{y}_c)x$. At inference the arg-max raw score decides the class. Trained with $\eta = 0.1$ for 100 epochs, reshuffling each epoch; the epoch with the best validation accuracy is retained.

Advantages: extremely simple, no gradients, diagnostic update-count per epoch. Limitations: step activation gives no error magnitude; one linear boundary per class cannot capture non-linear pixel-space structure; never converges on non-separable data.

Model B: Multi-Layer Perceptron (back-propagation)

A fully-connected feed-forward network maps the 1024-d input through one or more hidden layers with non-linear activations to 62 output logits (softmax inside the loss). Trained by back-propagation with a first-order optimizer; best-validation epoch is checkpointed and restored.

Components varied: activations (ReLU/kaiming-normal init, sigmoid & tanh/xavier-uniform init, zero bias); losses (cross-entropy on logits, squared error on softmax outputs); optimizers (gd = full-batch, sgd = mini-batch, sgd_momentum with $\mu = 0.9$, adam); dropout in $\{0, 0.2, 0.3, 0.5\}$; L2 weight decay in $\{0, 10^{-5}, 10^{-4}, 10^{-3}\}$.

7 Experimental Setup

Item	Value
Language	Python 3
Key libraries	NumPy, scikit-learn, pandas, Pillow, Matplotlib
Hardware	CPU
Random seed	42 (Python, NumPy)
Tuning budget	24 random-search trials $\times$ 30 epochs; greedy stage-wise search at 30 epochs; final training 1

8 Tuning Strategy Summary

Random search (24 trials) samples the full joint space (hidden architecture, activation, loss, optimizer, learning rate, batch size) to give an unbiased estimate of which hyper-parameters matter most, measured as the spread of mean validation accuracy across the levels of each hyper-parameter. Greedy stage-wise search then starts from the best random-search configuration and tunes optimizer $\times$ learning rate jointly first (since they interact strongly), then batch size, activation, loss, architecture/depth, dropout and weight decay, one group at a time, carrying the best setting forward. Final training uses the resulting configuration for 100 epochs with best-validation checkpointing.

9 Hyperparameter Selection

(a) Random Search — Hyper-parameter Impact

Hyper-parameter	Mean val-acc spread	Best level	Worst level
hidden	0.1996	512	128
lr	0.1557	0.001	0.0001
loss	0.1538	cross_entropy	mse
optimizer	0.1311	adam	gd
batch_size	0.1301	128	256
activation	0.1137	tanh	sigmoid

Top random-search configurations (of 24 trials, 30 epochs each):

Optimizer	lr	Batch	Activation	Loss	Hidden	Val Acc
sgd_momentum	0.001	32	relu	cross_entropy	256-128-64	0.4227
adam	0.01	64	relu	cross_entropy	512	0.4129
sgd_momentum	0.001	32	tanh	cross_entropy	256	0.3855
adam	0.0001	128	tanh	cross_entropy	256	0.3464
sgd	0.1	128	tanh	mse	512-256	0.3033
gd	0.1	16	tanh	cross_entropy	256	0.2485

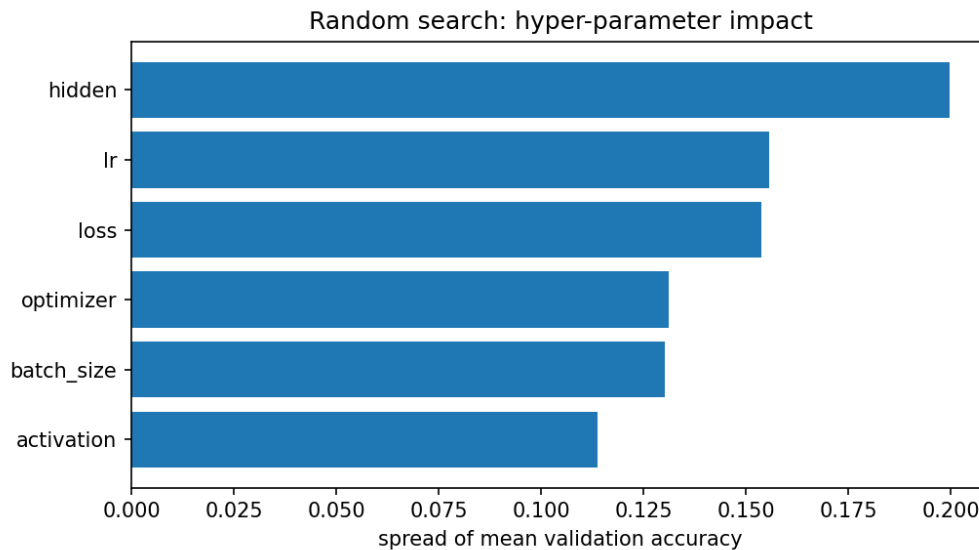


Figure 2: Random search — hyper-parameter impact

Best random-search configuration: hidden=(256-128-64), activation=relu, loss=cross_entropy, optimizer=sgd_momentum, lr=0.001, batch size=32, dropout=0.0, weight decay=0.0 (validation accuracy 0.4227). This was the starting point for the greedy stage-wise search.

Inference: architecture showed the widest marginal-mean spread (0.1996), with 512-unit hidden layers well ahead of 128-unit ones, closely followed by learning rate, loss function, optimizer and batch size. Full-batch GD and sigmoid activation were consistently the weakest settings, and cross-entropy clearly beat squared-error. Several trials — mostly combinations of a $1e-4$ learning rate or full-batch GD — sat near the $1/62 \approx 0.016$ chance level after only 30 epochs; these are configurations that had barely started training within the tuning budget, not configurations that are inherently unusable, as the stage-wise search below confirms once epochs and the optimizer/lr pairing are fixed.

(b) Greedy Stage-wise Search

Stage 1 — optimizer × learning rate (architecture/activation/loss/batch held at the best random-search config):

Optimizer	lr=1e-4	lr=1e-3	lr=1e-2	lr=1e-1
gd	0.0196	0.0196	0.0235	0.0528
sgd	0.0235	0.0450	0.4247	0.5068
sgd_momentum	0.0431	0.4227	0.5029	0.0626
adam	0.4070	0.4932	0.2798	0.0176

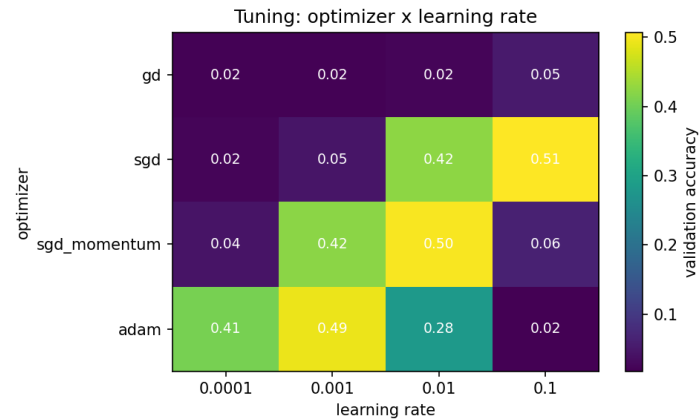
→ best: optimizer=sgd, lr=0.1

Stage 2 — batch size:

16	32	64	128	256
0.4012	0.5068	0.4990	0.4716	0.4247

→ best: batch size=32 (0.5068)

Stage 3 — activation:

Figure 3: Validation accuracy heat-map over the optimizer $\times$ learning-rate grid

relu	sigmoid	tanh
0.5068	0.1350	0.4266

→ best: relu (retained)

Stage 4 — loss function:

cross_entropy	mse
0.5068	0.3816

→ best: cross_entropy (retained)

Stage 5 — architecture / depth:

Hidden layers	Val accuracy
128	0.4501
256	0.4599
512	0.4716
256-256	0.5049
256-256-256	0.5127
256-256-256-256	0.5147
512-256-128	0.5264

→ best: 512-256-128 (0.5264)

Stage 6 — dropout:

0.0	0.2	0.3	0.5
0.5264	0.5342	0.4932	0.2857

→ best: dropout=0.2 (0.5342)

Stage 7 — weight decay (L2):

0.0	1e-05	0.0001	0.001
0.5342	0.5284	0.5362	0.5440

→ best: weight decay=0.001 (0.5440)

Final MLP configuration: hidden=(512-256-128), activation=relu, loss=cross_entropy, optimizer=sgd, lr=0.1, batch size=32, dropout=0.2, weight decay=0.001, trained for 100 epochs with the best-validation checkpoint restored (best validation accuracy 0.5812 at epoch 74).

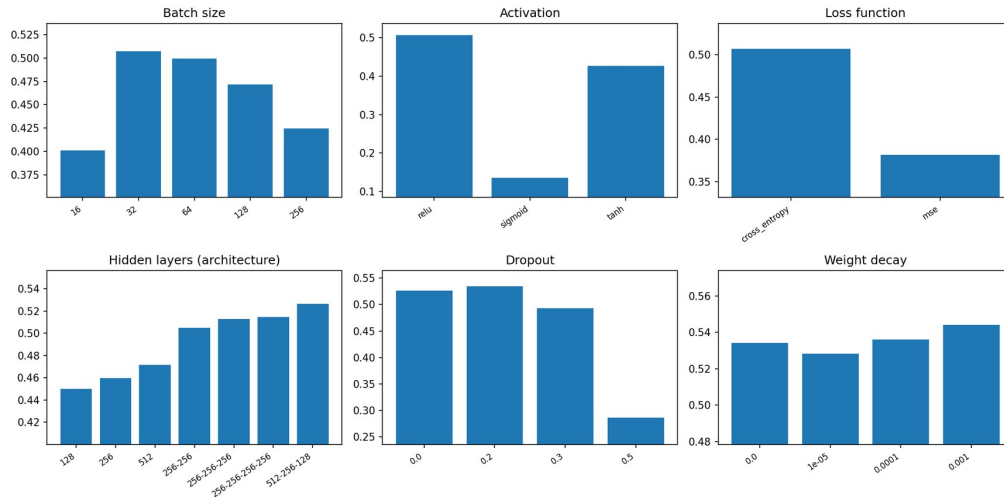


Figure 4: Validation accuracy per level within each greedy tuning stage

10 Convergence Results

Model A — PLA convergence

Epoch	Train error	Validation error	Samples triggering an update
1	0.8869	0.9315	2374
10	0.4801	0.8571	1895
20	0.3712	0.8219	1601
30	0.3159	0.8395	1432
40	0.2329	0.8258	1277
50	0.2061	0.8180	1188
60	0.1940	0.8376	1042
70	0.1487	0.8297	949
80	0.1382	0.8337	840
90	0.1093	0.8258	785
100	0.1106	0.8102	656

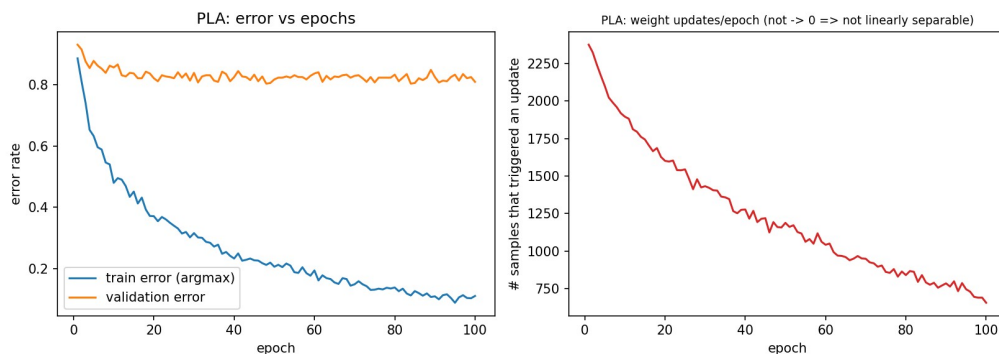


Figure 5: PLA — train/val error vs epochs (left); samples triggering an update per epoch (right)

Inference: training error falls steadily from 0.8869 to 0.1106, but validation error barely moves ($0.9315 \rightarrow 0.8102$), i.e. a validation accuracy of only about 19%. The decisive diagnostic is the update counter: 656 of 2387 training samples still triggered a weight update in the final epoch, decaying slowly rather than reaching zero. By the perceptron convergence theorem, a linearly

separable problem would drive this to zero in finite time — it does not, proving the 62 classes are not linearly separable in raw 1024-d pixel space. The large train-validation gap shows PLA is memorising individual training images through its 62 independent linear boundaries rather than learning a rule that generalises.

Model B — MLP convergence

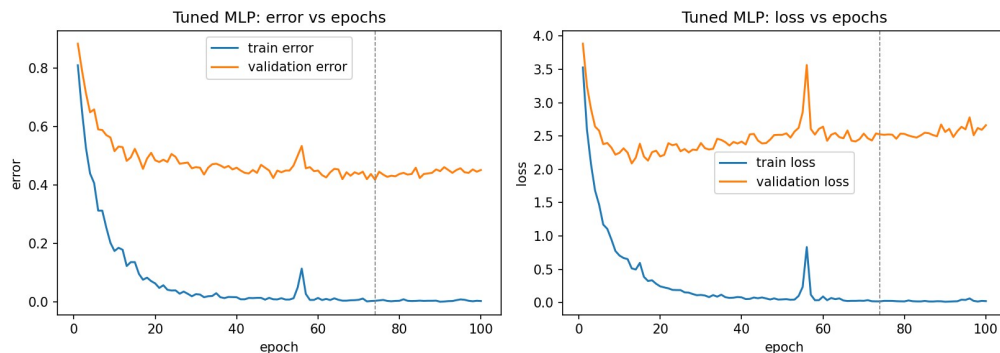


Figure 6: Tuned MLP — error (left) and loss (right) vs epochs, train and validation

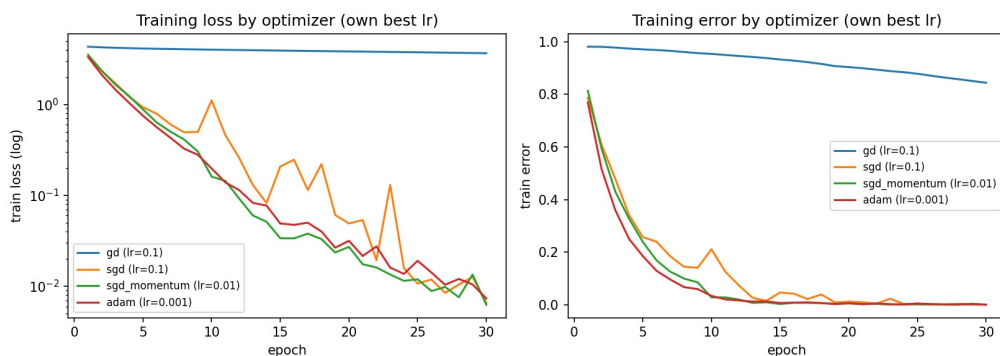


Figure 7: Training loss (log scale) and training error vs epochs for each optimizer at its own best learning rate

Inference: the tuned MLP (SGD, $lr=0.1$, $batch=32$, $dropout=0.2$, $weight\ decay=1e-3$) reached its best validation accuracy of 0.5812 at epoch 74 of 100; training accuracy at that checkpoint was 0.9958, so regularisation (dropout + weight decay) kept the model useful well beyond the 30-epoch tuning budget instead of collapsing into pure memorisation immediately. In the optimizer-convergence comparison, Adam drives training loss down fastest and most smoothly on a log scale; SGD and SGD-with-momentum need a well-chosen learning rate to compete but do so once it is found; full-batch GD is the slowest by a wide margin because it makes only one parameter update per epoch.

11 Results

Metric (test set, 512 images)	Model A: PLA	Model B: Tuned MLP
Accuracy	0.1895	0.543
Precision (macro)	0.2167	0.5615
Recall (macro)	0.1891	0.5441
F1 (macro)	0.1853	0.536
Precision (weighted)	0.2182	0.562
Recall (weighted)	0.1895	0.543
F1 (weighted)	0.1862	0.5356
AUC (micro, one-vs-rest)	0.7493	0.9427
AUC (macro, one-vs-rest)	0.753	0.9418

Reference points: random guessing over 62 near-balanced classes gives ~ 0.016 accuracy. The best validation accuracy reached during tuning was 0.5812.

Overfitting study (final tuned configuration, with and without dropout/weight decay):

Configuration	Train acc at best epoch	Val acc
Tuned (dropout=0.2, wd=1e-3)	0.9958	0.5812
No regularisation (dropout=0, wd=0)	0.9941	0.5264

12 Result Visualization

1. Confusion Matrices

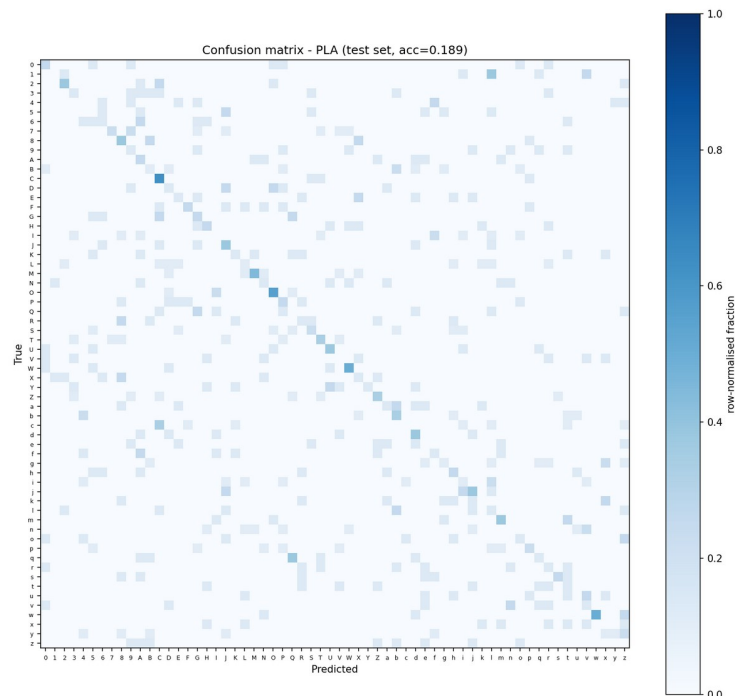


Figure 8: Row-normalised confusion matrix — PLA (test set)

Inference: the PLA confusion matrix shows weak, scattered diagonal mass consistent with its 0.1895 accuracy; its worst confusions are the classic look-alike pairs — q/Q , $1/l$, c/C , $8/X$ — where a single linear boundary genuinely cannot separate the pair. The tuned MLP shows

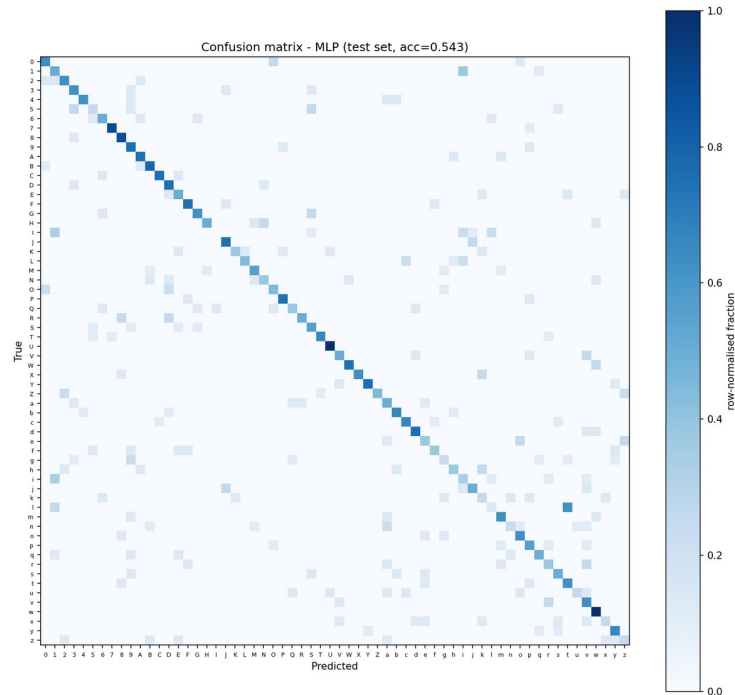


Figure 9: Row-normalised confusion matrix — tuned MLP (test set)

a much stronger diagonal (accuracy 0.543); its residual confusion is concentrated in visually near-identical strokes such as *l/t*, *1/i*, *i/1*, *I/1*, which are close to indistinguishable at 32×32 resolution regardless of model.

2. ROC Curves (one-vs-rest)

Inference: PLA's AUC (micro=0.7493, macro=0.753) is far higher than its raw accuracy (0.1895) would suggest, because AUC only needs the correct class to be ranked above incorrect ones by the raw decision score — informative ranking survives even when the arg-max decision is wrong. The MLP's AUC (micro=0.9427, macro=0.9418) is higher still and is computed from calibrated softmax probabilities rather than raw uncalibrated scores, so it is the fairer of the two comparisons; both models rank far above the chance diagonal.

3. A/B Metric Comparison

4. Overfitting Comparison

Inference: the train-validation accuracy gap is 0.4146 for the tuned (regularised) run at its best epoch (epoch 74) versus 0.4677 for the unregularised run, which reached its best validation epoch far earlier (epoch 18) before over-fitting set in. Regularisation therefore both delayed the point of over-fitting and — via the best-checkpoint restore — recovered a materially higher validation and test accuracy (test: 0.543 vs. 0.5215).

13 Discussion

1. Why does PLA underperform compared to MLP?

PLA reached a test accuracy of 0.1895 against 0.543 for the tuned MLP — a $2.87 \times$ improvement. Three causes are visible in the run. **Separability:** PLA was still updating weights on 656 training samples in its final epoch, so the perceptron convergence guarantee does not apply —

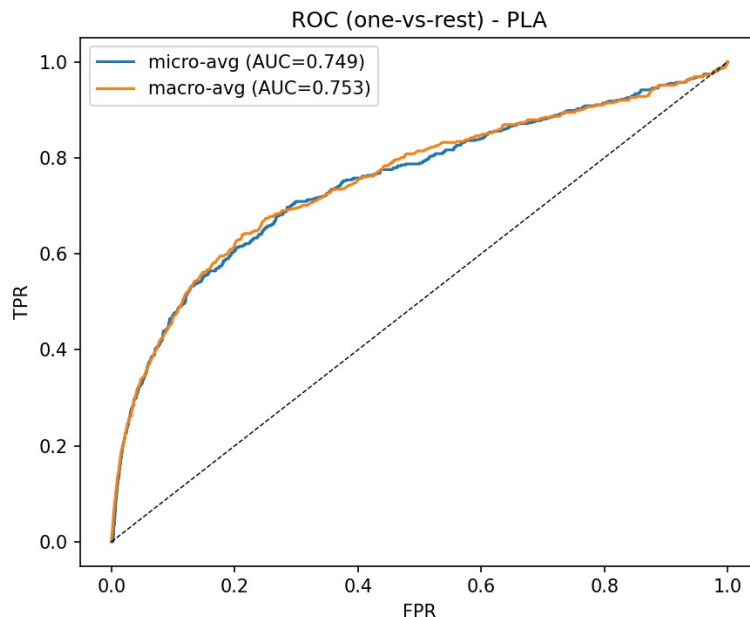


Figure 10: ROC curve (one-vs-rest) — PLA

the 62 classes are not linearly separable in pixel space, and no amount of additional epochs would fix that. **Representation:** PLA holds exactly one linear boundary per class operating directly on raw pixels, so two images of the same character with different slant or stroke width occupy distant regions of input space and cannot be covered by a single half-space; the MLP’s hidden layers learn intermediate features that collapse this variation before the final linear decision. **Learning signal:** the step activation is non-differentiable and emits only a binary error, whereas cross-entropy with softmax gives a graded gradient that scales with confidence and lets back-propagation credit-assign through the hidden layers.

2. Which hyper-parameters had the most impact?

On random search, hidden had the largest marginal spread (0.1996), followed by lr (0.1557), loss (0.1538), optimizer (0.1311), batch_size (0.1301) and activation (0.1137). The stage-wise search tells a different story once other settings are already reasonable: the optimizer $\times$ learning-rate stage spanned 0.0176–0.5068 (by far the widest range at that point), while later stages (batch size, loss, activation) spanned only a few points. The honest reading is that learning rate and optimizer dominate when a configuration is badly set, while architecture, dropout and weight decay provide the remaining gains once training is already working — and the random-search ranking likely overstates architecture somewhat, since wide-network trials happened to coincide with better optimizer/lr draws in this 24-trial sample.

3. Did optimizer choice affect convergence?

Substantially. Adam was the most robust across learning rates (val acc 0.407, 0.4932, 0.2798, 0.0176 at lr=1e-4..1e-1), because its per-parameter adaptive step sizes absorb a poorly chosen global rate. Plain SGD was far more rate-sensitive — it collapsed at low learning rates but achieved the single best result of the whole grid (0.5068 at lr=0.1), beating Adam’s best in this run. SGD with momentum peaked at a middling learning rate and became unstable at the highest one, the expected signature of an effective step size inflated by the momentum term. Full-batch GD was worst at every setting (max 0.0528), an artefact of update budget rather than the algorithm — with batch size equal to the full training set, it makes only one parameter update per epoch (30 updates total over the 30-epoch tuning budget) against roughly $75\times$ more

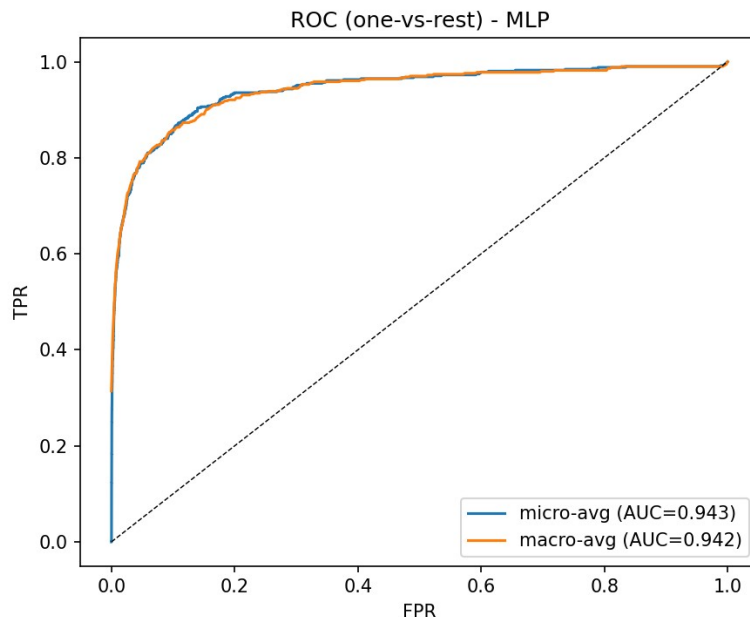


Figure 11: ROC curve (one-vs-rest) — tuned MLP

updates for batch-32 SGD over the same epochs.

4. Did adding more hidden layers always improve results?

No — but the picture here differs from a simple “wider always wins, deeper always loses” story. Validation accuracy across the depth stage was 128: 0.4501, 256: 0.4599, 512: 0.4716, 256-256: 0.5049, 256-256-256: 0.5127, 256-256-256-256: 0.5147, 512-256-128: 0.5264. Accuracy actually rose monotonically from a single 128-unit layer up through a 512-256-128 three-layer network, the best of the stage — because these architectures were tested downstream of a strong optimizer/lr choice (SGD, lr=0.1) that could exploit the extra capacity. With only 2387 training images across 62 classes (~ 38 per class), capacity still has to be spent carefully: within the dropout and weight-decay stages that followed, additional regularisation was needed precisely because the wider/deeper 512-256-128 network could otherwise memorise the training set (see the overfitting study).

5. Did the MLP overfit, and how was it mitigated?

Yes, characteristically. The tuned (regularised) run reached train accuracy 0.9958 vs. validation accuracy 0.5812 at its best epoch (74) — a gap of 0.4146. The same configuration without dropout/weight decay reached train accuracy 0.9941 vs. validation accuracy 0.5264 at a much earlier best epoch (18) — a gap of 0.4677, and a lower test accuracy (0.5215 vs. 0.543). Mitigations actually used: best-validation checkpointing (early stopping), dropout, and L2 weight decay, each with its own tuning stage. Further mitigations not used here: data augmentation (small rotations/translations, natural for handwriting), a convolutional architecture sharing weights across spatial positions, label smoothing, and more samples per class.

6. General observations on the achievable ceiling.

Even the best tuned configuration only reached 0.5812 validation accuracy. This reflects the setup rather than a tuning failure: a fully-connected network on flattened 32×32 pixels has no translation invariance, 62 classes include intrinsically ambiguous upper/lower-case pairs that are unresolvable without context, and ~ 55 images per class is very little data. A convolutional network with augmentation would be the natural next step and should move this ceiling

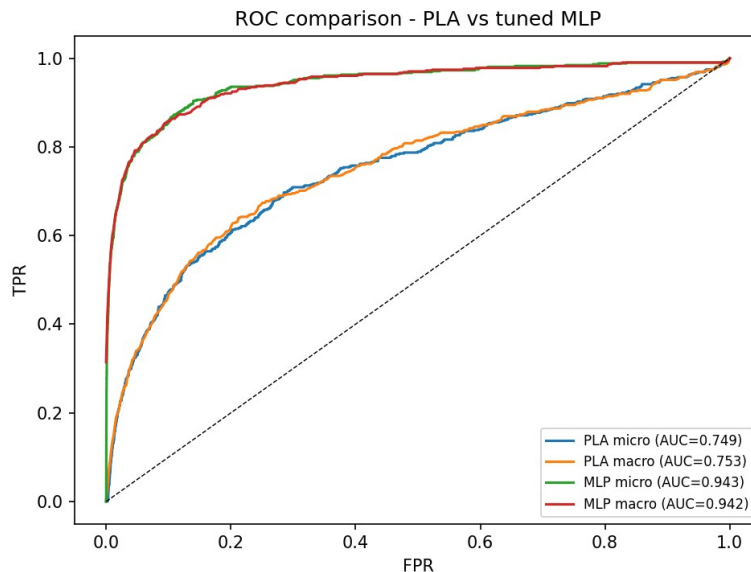


Figure 12: ROC comparison — PLA vs. tuned MLP, micro and macro averaged

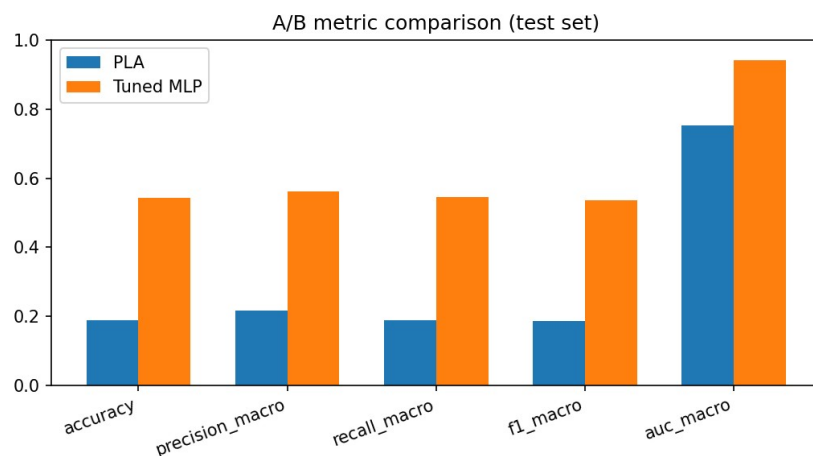


Figure 13: Accuracy, macro precision, macro recall, macro F1 and macro AUC — PLA vs. tuned MLP

substantially.

14 Conclusion

This experiment compared a from-scratch one-vs-rest Perceptron Learning Algorithm against a systematically tuned Multi-Layer Perceptron on 3410 handwritten English character images across 62 classes, using a stratified 70/15/15 split and 32×32 standardized grayscale pixels as input. PLA achieved 18.95% test accuracy and, critically, never stopped updating weights (656 of 2387 samples still triggered updates in epoch 100), empirically demonstrating that the problem is not linearly separable and that a single-layer perceptron with step activation is structurally incapable of solving it. The MLP was tuned first by 24-trial random search — which identified architecture, learning rate and loss function as the highest-impact hyper-parameters and produced a best configuration of a (256-128-64)-unit ReLU network with sgd-momentum at $lr=0.001$ (0.4227 validation accuracy) — and then by greedy stage-wise search, which im-

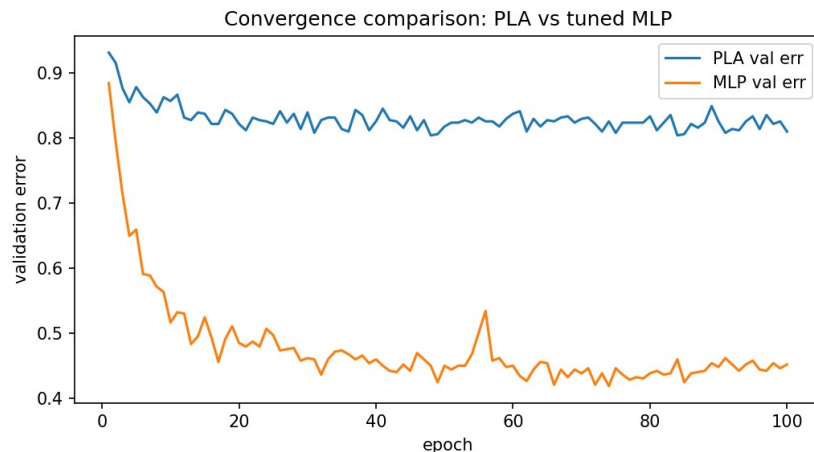


Figure 14: Convergence comparison — PLA vs. tuned MLP validation error

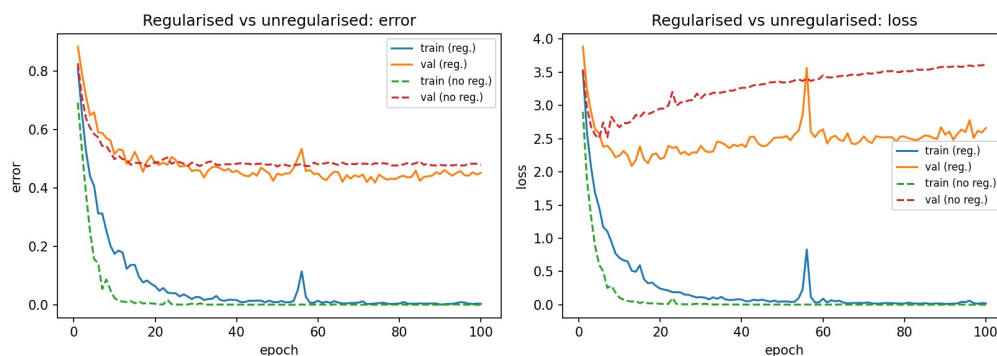


Figure 15: Train vs. validation error/loss, with and without regularisation

proved this to 0.5812 with a (512-256-128) ReLU network, sgd at $\text{lr}=0.1$, batch size 32, dropout 0.2 and weight decay 0.001. Optimizer behaviour was a dominant factor: Adam was robust across learning rates, plain SGD was rate-sensitive but produced the single best grid result once correctly tuned, and full-batch GD was starved of updates under a fixed epoch budget. The final tuned MLP achieved 54.30% on the held-out test set, $2.87\times$ the PLA baseline, confirming that non-linear feature learning through hidden layers — not merely additional parameters — is what makes this task tractable.

15 References

- [1] Rosenblatt, F., “The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain,” *Psychological Review*, vol. 65, no. 6, pp. 386–408, 1958.
- [2] Minsky, M. and Papert, S., *Perceptrons: An Introduction to Computational Geometry*, MIT Press, 1969.
- [3] Rumelhart, D. E., Hinton, G. E. and Williams, R. J., “Learning representations by back-propagating errors,” *Nature*, vol. 323, pp. 533–536, 1986.
- [4] Kingma, D. P. and Ba, J., “Adam: A Method for Stochastic Optimization,” *ICLR*, 2015.
- [5] Srivastava, N. et al., “Dropout: A Simple Way to Prevent Neural Networks from Overfitting,” *JMLR*, vol. 15, pp. 1929–1958, 2014.

- [6] Glorot, X. and Bengio, Y., “Understanding the difficulty of training deep feedforward neural networks,” *AISTATS*, 2010, pp. 249–256.
- [7] He, K. et al., “Delving Deep into Rectifiers,” *ICCV*, 2015, pp. 1026–1034.
- [8] Bergstra, J. and Bengio, Y., “Random Search for Hyper-Parameter Optimization,” *JMLR*, vol. 13, pp. 281–305, 2012.
- [9] Dave, D., *English Handwritten Characters Dataset*, Kaggle: dhruvildave/english-handwritten-characters-dataset.
- [10] Paszke, A. et al., “PyTorch: An Imperative Style, High-Performance Deep Learning Library,” *NeurIPS* 32, 2019.